

Programare Orientată Pe Obiecte

Curs 7 - partea 1
Polimorfism recapitulare
Interfețe
Interfața Comparable
Exerciții

Noiembrie 2025

Recapitulare Polimorfism

Metode in JAVA

Structura unei Metode:

- Metoda este un bloc de cod cu o funcție specifică.
- Structura:
 - `modifier_acces tip_retur numeMetoda(parametri)`
`{ ... }`

Metode in JAVA

Structura unei Metode:

- Metoda este un bloc de cod cu o funcție specifică.
- Structura:
 - `modifier_acces tip_retur numeMetoda(parametri)`
`{ ... }`

Exemplu

```
public int aduna(int a, int b) { return a + b; }
```

Tipuri de Metode:

- **Metode instanță:** apelate pe obiecte instanțiate.
- **Metode statice:** apelate direct pe clasă, fără instanțierea unui obiect.

Suprascrierea și Supraincarea Metodelor

- **Suprascriere (Override):** rescrierea unei metode moștenite:

```
class Animal {  
    public void sunet() {  
        System.out.println("Animalul scoate un sunet");  
    }  
}  
  
class Caine extends Animal {  
    @Override public void sunet() {  
        System.out.println("Cainele latra");  
    }  
}
```

- **Supraincercare (Overload):** definirea mai multor metode cu același nume, dar parametri diferiți:

```
public int inmulteste(int a, int b) { return a * b; }  
public double inmulteste(double a, double b) { return a * b; }
```

Polimorfism în Java

- Permite ca un obiect să ia mai multe forme.
- Scrierea unui cod generic și flexibil.
- Două tipuri:
 - Polimorfism la timp de compilare.
 - Realizat prin supraîncărcarea metodelor.
 - Metoda corectă este aleasă la compilare, pe baza parametrilor.
 - Apel de metodă sau constructor cu **super()**
 - Polimorfism la timp de execuție:
 - Realizat prin suprascrierea metodelor.
 - Metoda este aleasă la rulare în funcție de tipul real al obiectului.
 - **this()**...

Polimorfism în Java

- Permite ca un obiect să ia mai multe forme.
- Scrierea unui cod generic și flexibil.
- Două tipuri:
 - Polimorfism la timp de compilare.
 - Realizat prin supraîncărcarea metodelor.
 - Metoda corectă este aleasă la compilare, pe baza parametrilor.
 - Apel de metodă sau constructor cu **super()**
 - Polimorfism la timp de execuție:
 - Realizat prin suprascrierea metodelor.
 - Metoda este aleasă la rulare în funcție de tipul real al obiectului.
 - **this()**...

Exemplu

```
Animal myAnimal = new Caine();  
myAnimal.sunet(); // Apelează sunet() din Caine
```

Metode Polimorfe

```
public class Person {  
    public void method1() {  
        System.out.println("Person 1 ");  
    }  
    public void method2() {  
        System.out.println("Person 2 ");  
    }  
}  
  
class Student extends Person {  
    public void method1() {  
        System.out.println("Student 1 ");  
        super.method1();  
        method2(); //this.method2();  
    }  
    public void method2() {  
        System.out.println("Student 2 ");  
    }  
}  
  
class Undergrad extends Student {  
    public void method2() {  
        System.out.println("Undergrad 2 ");  
    }  
}
```

- Ce se va afișa la executarea următoarelor linii de cod?

```
Person p = new Undergrad();  
//Person p = new Student();  
p.method1();
```

Rezultate afișate:

Student 1
Person 1
Undergrad 2

Crearea obiectelor: ordinea inițializărilor când nu avem moștenire

- În Java, ordinea inițializării variabilelor și blocurilor la crearea unui obiect este strict definită.
- Când un obiect este creat (**la invocarea operatorului new**), Java respectă următoarea ordine de inițializare (presupunând că nu există moștenire):
 - ❶ Variabile statice și blocuri statice: sunt inițializate o singură dată, la încărcarea clasei, înainte de a crea orice obiect al acelei clase.
 - ❷ Variabile de instanță și blocuri de instanță: sunt inițializate de fiecare dată când se creează un nou obiect al clasei.
 - ❸ Constructorul: este apelat după ce toate variabilele de instanță și blocurile de instanță au fost inițializate.

Crearea obiectelor: ordinea inițializărilor când nu avem moștenire

```
class Exemplu{
    static int variabilaStatika = initializeazaVariabilaStatika();
    static { System.out.println("Bloc static"); }
    int variabilaInstanta = initializeazaVariabilaInstanta();
    {
        System.out.println("Bloc de instanta");
    }
    public Exemplu() { System.out.print("Constructor"); }
    // Metoda de initializare pentru variabila statica
    private static int initializeazaVariabilaStatika() {
        System.out.println("Initializare variabila statica");
        return 10;
    }
    // Metoda de initializare pentru variabila de instanta
    private int initializeazaVariabilaInstanta() {
        System.out.println("Initializare variabila de instanta");
        return 5;
    }
}
```

Crearea obiectelor: ordinea inițializărilor când nu avem moștenire

Ce se va afișa la rularea codului?

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Creare primul obiect:");  
        Exemplu primulObiect = new Exemplu();  
        System.out.println("\nCreare al doilea obiect:");  
        Exemplu alDoileaObiect = new Exemplu();  
    }  
}
```

Crearea obiectelor: ordinea inițializărilor când nu avem moștenire

Ce se va afișa la rularea codului?

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Creare primul obiect:");  
        Exemplu primulObiect = new Exemplu();  
        System.out.println("\nCreare al doilea obiect:");  
        Exemplu alDoileaObiect = new Exemplu();  
    }  
}
```

Initializare variabila statica

Bloc `static`

Creare primul obiect:

Initializare variabila de instanta

Bloc de instanta

Constructor

Creare al doilea obiect:

Initializare variabila de instanta

Bloc de instanta

Constructor

Crearea obiectelor și ordinea inițializărilor - dacă nu avem moștenire

Ce se afișează la rularea codului, când doar declarăm obiectele:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Declarare primul obiect:");  
        Exemplu primulObiect;  
        System.out.println("\nDeclarare al doilea obiect:");  
        Exemplu alDoileaObiect;  
    }  
}
```

Crearea obiectelor și ordinea inițializărilor - dacă nu avem moștenire

Ce se afișează la rularea codului, când doar declarăm obiectele:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Declarare primul obiect:");  
        Exemplu primulObiect;  
        System.out.println("\nDeclarare al doilea obiect:");  
        Exemplu alDoileaObiect;  
    }  
}
```

Se va afisa: Declarare primul obiect: Declarare al doilea obiect:

Ce se afișează la rularea codului, la invocarea operatorului new:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Create primul obiect:");  
        Exemplu primulObiect = new Exemplu();  
        System.out.println("\nCreate al doilea obiect:");  
        Exemplu alDoileaObiect = new Exemplu();  
    }  
}}
```

Crearea obiectelor și ordinea inițializărilor - dacă avem moștenire

Ordinea execuției când avem moștenire este următoarea:

- 1 Inițializarea câmpurilor statice și a blocurilor statice ale clasei părinte (o singură dată, când clasa părinte este încărcată).
- 2 Inițializarea câmpurilor statice și a blocurilor statice ale clasei derivate (o singură dată, când clasa derivată este încărcată).
- 3 Inițializarea câmpurilor de instanță și a blocurilor de inițializare a instanței din clasa părinte (pentru fiecare instanță creată).
- 4 Executarea constructorului clasei părinte.
- 5 Inițializarea câmpurilor de instanță și a blocurilor de inițializare a instanței din clasa derivată.
- 6 Executarea constructorului clasei derivate.

Crearea obiectelor și ordinea inițializărilor - dacă avem moștenire

```
public abstract class Forma {
    static String background = "transparent";
    String culoare;
    abstract void draw();
    Forma(String c){
        System.out.println("Forma inainte de draw");
        culoare = c;
        draw();
        System.out.println("Forma dupa draw");  } }
public class FormaRotunda extends Forma{
    private int raza = 1;
    private static int tipLinie = 3;
    FormaRotunda(int r, String c){
        super(c);
        raza = r;
        System.out.println("Constructor FormaRotunda raza = " + raza +
            " tipLinie = " + tipLinie + " background: " + background +
            " culoare:" + culoare);  }
    void draw() {
        System.out.println("Draw FormaRotunda raza = " + raza +
            " tipLinie = " + tipLinie + " background: " + background +
            " culoare:" + culoare);  } }
```

Crearea obiectelor și ordinea inițializărilor - dacă avem moștenire

```
public class Main {  
    public static void main(String[] args){  
        new FormaRotunda(5, "verde");  
    }  
}
```

Se va afișa:

Forma inainte de draw!

Draw FormaRotunda raza = 0 tipLinie = 3 background: transparent
culoare:verde

Forma dupa draw!

Constructor FormaRotunda raza = 5 tipLinie = 3 background: transparent
culoare:verde

Constructorii și polimorfismul

- Constructorul are un rol special: trebuie sa se asigure ca obiectul este construit corect.
- O clasa derivata are acces (de obicei) doar la membrii săi și nu la membrii clasei părinte (care sunt de obicei private). Doar constructorul clasei părinte are acces la membri private si “știe” să îi initializeze corect.
- Asadar este esential sa se execute constructorii claselor parinte
- Compilatorul “obligă” chemarea unui constructor din clasa parinte pentru orice constructor al unei clase copil !
- Dacă o clasă părinte nu are constructor se va chema “ silențios” constructorul implicit din clasa părinte.

Ordinea executiei constructorilor

Ce se va afișa la rulare?

```
class Mancare {
    Mancare() { System.out.println("Mancare()"); } }
class Paine {
    Paine() { System.out.println("Paine()"); }}
class Cascaval {
    Cascaval() { System.out.println("Cascaval()"); } }
class Salata {
    Salata() { System.out.println("Salata()"); }}
class Pranz extends Mancare {
    Pranz() { System.out.println("Pranz()"); } }
class PranzaLaPachet extends Pranz {
    PranzaLaPachet() { System.out.println("PranzaLaPachet()");
    } }
class Sandwich extends PranzaLaPachet {
    Paine b = new Paine();
    Cascaval c = new Cascaval();
    Salata s = new Salata();
    Sandwich() { System.out.println("Sandwich()"); }
}
```

Ordinea executiei constructorilor

Ce se va afișa la rulare?

```
public class Main {  
    public static void main(String[] args) {  
        new Sandwich();  
    }  
}
```

Ordinea executiei constructorilor

Ce se va afișa la rulare?

```
public class Main {  
    public static void main(String[] args) {  
        new Sandwich();  
    }  
}
```

Se va afișa:

```
Mancare()  
Pranz()  
PranzaLaPachet()  
Paine()  
Cascaval()  
Salata()  
Sandwich()
```

Quizz ordinea initializarilor

Interfețe

Specificațiile și Java

- Un program este asamblat dintr-o colecție de clase care trebuie să "lucreze împreună" sau să "se potrivească una cu alta"
- Limbajul și compilatorul Java ne pot ajuta să:
 - Scriem specificații de clase și să
 - Verificăm că o clasă satisface corect (implementează) specificațiile sale
- Clase concrete, clase abstracte și interfețe:
 - 1 construcția **extends** - ne permite să codificăm o clasă prin adăugarea de metode la o clasă existentă
 - 2 construcția **abstract class** - ne permite să codificăm o clasă incompletă care poate fi încheiată (terminată) printr-o altă clasă
 - 3 construcția **interface** - ne permite să codificăm în Java informația pe care o specificăm, d.e. într-o diagramă de clasă

Ce este o interfață

Java: o interfață este un tip, așa cum și o clasă este un tip:

- Asemănător unei clase, o interfață definește metode
- Spre deosebire de o clasă, o interfață nu implementează niciodată metode (valabil în Java SE 7 sau versiuni mai vechi), dar în Java 8 avem metode default
- Clasele care implementează interfața implementează metodele definite de interfață
- O clasă poate implementa mai multe interfețe

O interfață se folosește pentru a defini un protocol de comportament care poate fi implementat de către orice clasă de oriunde din ierarhia de clase

Definiție și utilitate interfețe:

- **Definiție:** o interfață este o colecție de constante si definiții de metode, (fără implementări), colecție care are un nume
- O interfață nu este o clasă, ci un set de **cerințe** pentru clasele care doresc să se conformeze interfeței
- Interfețele sunt folositoare pentru:
 - Reținerea asemănărilor între clase ne-înrudite fără a forța o relație de clasă
 - Declararea de metode pe care una sau mai multe clase ar trebui să le implementeze
 - Dezvăluirea interfeței de programare a unui obiect fără a-i dezvălui clasa
 - Modelarea moștenirii multiple, care permite ca o clasă să aibă mai mult de o superclasă

Sintaxa JAVA pentru definirea și implementarea unei interfețe

Definirea unei interfețe are două componente: declarația interfeței și corpul interfeței

- Declarația interfeței definește diferitele atribute ale interfeței, cum sunt numele și dacă ea extinde alte interfețe
- Corpul interfeței conține declarațiile de constante și de metode pentru interfața respectivă

```
[modificator acces] interface NumeInterfata [extends AltaInterfata]{  
    /* zona declarare constante */  
    public static final int X = 10;  
    /*zona declarare metode*/  
    public void numeMetoda();  
}  
  
//Implementarea unei interfete:  
class NumeClasa implements NumeInterfata{  
    public void numeMetoda(){  
        // codul necesar pentru implementarea metodei  
    }  
}
```

Exemplu interfețe

```
class MobileDevice extends Device{
    int displaySize;
    MobileDevice(){
        System.out.println("Mobile Device Constructor");
    }
    MobileDevice(String manufacturer, String model, int displaySize){
        super(manufacturer, model);
        this.displaySize = displaySize;
        this.batteryLife = batteryLife;
    }
}

class DesktopPC extends Device{
    ArrayList<String> periferice; //mouse, tastatura, microfon etc
    DesktopPC(ArrayList<String> p){
        this.periferice = p;
    }
}

public class DemoDevice {
    public static void main(String[] args) {
        MobileDevice md = new MobileDevice();
        MobileDevice md2 = new MobileDevice("Samsung", "Galaxy S22", 6);
    }
}
```

Exemplu interfețele GPS și Radio

Creați clasa **SmartPhone** care are caracteristicile unui **MobileDevice** dar în plus are aplicații pt **GPS** și **Radio**!

O clasă poate implementa mai multe interfețe !

```
interface GPS {  
    public void getCoordinates();  
}  
  
interface Radio {  
    public void startRadio();  
    public void stopRadio();  
}  
  
class Smartphone extends MobileDevice implements GPS, Radio {  
    public void getCoordinates() {  
        System.out.println("GPS Coordinates for Smartphone");  
    }  
    public void startRadio() {  
        System.out.println("Start Radio for Smartphone");  
    }  
    public void stopRadio() {  
        System.out.println("Stop Radio for Smartphone");  
    }  
}
```

Proprietăți și observații interfețe:

Interfețele nu se pot instanția:

```
public class DemoDevice {  
    public static void main(String[] args) {  
        Radio r = new Radio(); // ERROR - Cannot instantiate type Radio !  
    }  
}
```

DAR se pot instanția clase care implementează interfețe:

```
public class DemoDevice {  
    public static void main(String[] args) {  
        Radio r = new Smartphone();  
    }  
}  
// Se va afisa:  
// Device constructor  
// Mobile Device Constructor
```

Proprietăți și observații interfețe:

- Toate variabilele și metodele dintr-o interfață sunt publice (chiar dacă nu este folosit cuvântul public la declarație).
- Interfețele nu pot specifica implementarea metodelor
- De la versiunea Java 8 se pot declara metode default.

```
interface GPS {  
    public void getCoordinates();  
    default public void getCoordinateWifi() {  
        // returneaza coordonatele de la sist wifi  
        System.out.println("Initializeaza coordonatele de la wifi...");  
    }  
}  
  
public class DemoDevice {  
    public static void main(String[] args) {  
        Smartphone sm = new Smartphone();  
        sm.getCoordinateWifi();  
        sm.startRadio();  
    }  
}  
  
/* Se afiseaza:  
Device constructor  
Mobile Device Constructor  
Initializeaza coordonatele de la wifi ...  
Start Radio for Smartphone */
```

Interfețe: coliziunea de nume la declarația metodelor default

Ce se intampla daca 2 interfete au aceeasi declaratie de metoda ?

```
interface Radio {  
    public void startRadio();  
    public void stopRadio();  
    default public void next() {  
        System.out.println("Next from  
            Radio");    } } }
```

```
interface MusicPlayer {  
    public void startMusicPlayer();  
    //public void stopMusicPlayer();  
    default public void next() {  
        System.out.println("Next from  
            MusicPlayer");    } } }
```

```
class Smartphone extends MobileDevice implements GPS, Radio,  
    MusicPlayer{  
    public void getCoordinates() {  
        System.out.println("GPS Coordinates for Smartphone");    }  
    public void startRadio() {  
        System.out.println("Start Radio for Smartphone");    }  
    public void stopRadio() {  
        System.out.println("Stop Radio for Smartphone");    }  
    public void startMusicPlayer() {  
        System.out.println("Start Music Player");    }  
}
```

EROARE: Duplicate default methods named next with the parameters () and () are inherited from the two super-interfaces MusicPlayer and Radio

Interfețe: coliziunea de nume la declarația metodelor default

Ce se intampla daca 2 interfete au aceeasi declaratie de metoda ?

Soluție 1: scriem implementarea metodei

```
class Smartphone extends MobileDevice implements GPS, Radio,
    MusicPlayer {
    public void getCoordinates() {
        System.out.println("GPS Coordinates for Smartphone"); }
    public void startRadio() {
        System.out.println("Start Radio for Smartphone"); }
    public void stopRadio() {
        System.out.println("Stop Radio for Smartphone"); }
    public void startMusicPlayer() {
        System.out.println("Start Music Player"); }
    public void next() {
        System.out.println("Next in smartphone");
    }
}

public class DemoDevice {
    public static void main(String[] args) {
        Smartphone sm = new Smartphone();
        sm.getCoordinateBrute();
        sm.startRadio();
```

Interfețe: coliziunea de nume la declarația metodelor default

Ce se intampla daca 2 interfete au aceeasi declaratie de metoda ?

Soluție 1: scriem implementarea metodei, si folosim `super` pt a alege metoda default din interfata dorita !

```
class Smartphone extends MobileDevice implements GPS, Radio,
    MusicPlayer {
    public void getCoordinates() {
        System.out.println("GPS Coordinates for Smartphone"); }
    public void startRadio() {
        System.out.println("Start Radio for Smartphone"); }
    public void stopRadio() {
        System.out.println("Stop Radio for Smartphone"); }
    public void startMusicPlayer() {
        System.out.println("Start Music Player"); }
    public void next() {
        MusicPlayer.super.next(); } }

public class DemoDevice {
    public static void main(String[] args) {
        Smartphone sm = new Smartphone();
        sm.getCoordinateBrute();
        sm.startRadio();
        sm.next();
    }
}
```

Metode statice in interfete (de la Java 8)

- Metodele statice din interfete nu sunt "moștenite" de clasele care implementeaza interfața.
- Cand se cheamă metodele statice se utilizează interfața și nu clasa care implementează interfața !

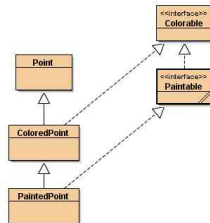
```
interface MusicPlayer {  
    public void startMusicPlayer();  
    //public void pauseMusicPlayer();  
    default public void next() { System.out.println("Next from  
        MusicPlayer"); }  
    public static void reclama(String sponsor) {  
        System.out.println("Reclama de la " + sponsor); }  
}  
  
public class DemoDevice {  
    public static void main(String[] args) {  
        Radio r = new Smartphone();  
        Smartphone sm = new Smartphone();  
        sm.getCoordinateBrute();  
        sm.startRadio();  
        sm.next();  
        MusicPlayer.reclama("Bosch"); // Folosim interfata nu clasa  
        // sm.reclama("Bosch"); // Produce eroare de compilare  
    } }
```

Ce putem pune într-o interfață (Java 8)

- 1 Constante (static final) de orice tip; Valorile se pot stabili la compilare sau se pot calcula la încărcarea clasei
- 2 Metode abstracte: metode fără implementare; Trebuie implementate de clasele care implementează interfața.
- 3 Metode default – sunt metode concrete (cu implementare) pentru care clasele care extind interfata nu mai trebuie sa dea o implementare.
- 4 Metode statice: ele nu sunt "moștenite" de clasele care implementeaza interfața; când se cheamă metodele statice se utilizează interfața și nu clasa care implementează interfața !

Superinterfețe

- Dacă este furnizată o clauză extends, atunci interfața în curs de declarare extinde fiecare dintre celelalte interfețe numite și din acest motiv moștenește metodele și constantele fiecăreia dintre celelalte interfețe numite
- Aceste alte interfețe sunt numite superinterfețe directe ale interfeței în curs de declarare
- Orice clasă care implementează - implements - interfața declarată se consideră că implementează toate interfețele pe care aceasta interfață le extinde și care sunt accesibile clasei



Interfețe, Moștenire și Coliziunea de Nume

```
interface I1 { void f(); }
interface I2 { int f(int i); }
interface I3 { int f(); }
class C {
    int f(){
        return 1;
    }
}
class C2 implements I1, I2{
    public void f(){}
    public int f(int i){ return 1;
    } //supraincarcata din I2
}
class C3 extends C implements I2 {
    public int f(int i){
        return 1;
    } //supraincarcata din I2
}
```

```
class C4 extends C implements I3 {
    public int f(){
        return 1;
    } // e identica - e ok
}
//metodele difera doar prin tipul
//returnat!
class C5 extends C implements I1 {
    public void f(){
    }
}
interface I4 extends I1, I3 { }
//eroare de compilare
```

```
//metodele difera doar prin tipul returnat!
class C5 extends C implements I1 {
    public void f(){
    }
}
interface I4 extends I1, I3 { }
//eroare de compilare
```

Comparable

Interfețe predefinite

Interfața Comparable

- Interfața Comparable este definită în pachetul java.lang, fiind automat disponibilă fiecărui program
- Are doar metodă care trebuie implementată: `public int compareTo(Object other);`
- Este responsabilitatea programatorului să urmeze semantica interfeței Comparable atunci când implementează.
- Metoda compareTo trebuie să returneze:
 - 1 Un număr negativ atunci când un obiect "este înainte de" parametrul other
 - 2 Zero atunci când obiectul apelant "este egal cu" parametrul other
 - 3 Un număr pozitiv dacă obiectul apelant "urmează după" parametrul other
- Semantica lui "equals" din metoda compareTo ar trebui să coincidă cu cea a metodei equals dacă se poate, dar aceasta nu este absolut necesar

Interfața Comparable: Exemplu

```
import java.util.*;
public class Person implements Comparable<Person>{
    private String nume, prenume;
    private int varsta;
    public Person(String n, String pr, int v) {
        nume = n;
        prenume = pr;
        varsta = v;
    }
    //@implements: compareTo(Object p) in Object
    public int compareTo(Person p)
    {
        return nume.compareTo(p.nume);
    }
    //@Overrides: toString() in Object
    public String toString(){
        return (nume+ " " + prenume + ", " + varsta + " ani"); }
}
```

Interfața Comparable: Exemplu

```
public class Test {  
    public static void afisare(Person[] v){  
        for (int i = 0; i<v.length; i++)  
            System.out.println(v[i].toString());  
    }  
    public static void main() {  
        Person[] v= new Person[3];  
        v[0] = new Person("Pop", "Ioan", 20);  
        v[1] = new Person("Ana", "Maria", 15);  
        v[2] = new Person("Popescu", "Daria", 18);  
        System.out.println("Vectorul initial de persoane:");  
        afisare(v);  
        Arrays.sort(v);  
        System.out.println("\nVectorul sortat dupa nume:");  
        afisare(v);  
    }  
}
```

Rezultatul afisat:

Vectorul initial de persoane: Pop Ioan, 20 ani ; Ana Maria, 15 ani;
 Popescu Daria, 18 ani

Vectorul sortat dupa nume: Ana Maria, 15 ani; Pop Ioan, 20 ani;
 Popescu Daria, 18 ani

Două metode de sortare, folosind interfața **Comparator**

Dar dacă doresc să sortez un vector după arie și alt vector după perimetru?
Se pot implementa comparatori diferiți folosind interfața **Comparator**:

```
Collections.sort(yourList, new CustomComparator<YourClass>());
```

Exemplu:

```
class CompareByArie implements Comparator<Shape>{
    @Override
    public int compare(Shape s, Shape t) {
        if (s.area() > t.area()) return 1;
        if (s.area() < t.area()) return -1;
        return 0;
    }
}

class CompareByPerimetru implements Comparator<Shape>{
    @Override
    public int compare(Shape s, Shape t) {
        if (s.perimetru() > t.perimetru()) return 1;
        if (s.perimetru() < t.perimetru()) return -1;
        return 0;
    }
}
```

Două metode de sortare, folosind interfața **Comparator**

Dar dacă doresc să sortez un vector după arie și alt vector după perimetru?
Se pot implementa comparatori diferiți folosind interfața **Comparator**:
Continuare exemplu:

```
public class Main {  
    public static void main(String[] args) {  
        ArrayList<Shape> lst = new ArrayList<Shape>();  
        lst.add(new Circle(5, "Red"));  
        lst.add(new Circle(15, "Blue"));  
        lst.add(new Square("Blue", 4));  
        for (Shape shape : lst) shape.afisareInfo();  
        Collections.sort(lst, new CompareByArie());  
        for (Shape shape : lst) shape.afisareInfo();  
        Collections.sort(lst, new CompareByPerimetru());  
        for (Shape shape : lst) shape.afisareInfo();  
    }  
}
```

Două metode de sortare, folosind interfața **Comparator** și clase anonime !

Nu este nevoie sa se creeze o clasă pentru fiecare situație de comparație !
Se pot implementa comparatori diferiți folosind interfața **Comparator** și clase anonime ! Exemplu:

```
public class Main {  
    public static void main(String[] args) {  
        ArrayList<Shape> lst = new ArrayList<Shape>();  
        lst.add(new Circle(5, "Red"));  
        lst.add(new Circle(15, "Blue"));  
        lst.add(new Square("Blue", 4));  
        Collections.sort(lst, new Comparator<Shape>() {  
            @Override  
            public int compare(Shape s, Shape t) {  
                if (s.area() > t.area()) return 1;  
                if (s.area() < t.area()) return -1;  
                return 0;  
            }  
        });  
    }  
}
```

Două metode de sortare, folosind interfața **Comparator** și clase anonime !

Dar dacă doresc să sortez un vector după arie și alt vector după perimetru?

Folosim **Comparator** !

Se pot implementa comparatori diferiți folosind interfața **Comparator**:

```
// sortare dupa perimetru
Collections.sort(lst, new Comparator<Shape>() {
    @Override
    public int compare(Shape s, Shape t) {
        if (s.perimetru() > t.perimetru()) return 1;
        if (s.perimetru() < t.perimetru()) return -1;
        return 0;
    }
});
}}
```

Rezumat: Interfețe, Clase Abstracte, Clase Concrete

Interfața

- Abstractă prin definiție;
- Separă implementarea unui obiect de specificarea sa ;
- Nu fixează nici un aspect al unei implementări

Clasa abstractă:

- Oferă o bază pe care să se construiască clase concrete

Clasa concretă:

- Completează implementarea efectivă a metodelor abstracte care au fost specificate de o interfață sau printr-o clasă abstractă
- Furnizează obiecte la momentul execuției
- Nu este, în general, potrivită ca bază pentru extindere

Exerciții

Forme Geometrice: Desenabile și Scalabile

Definiți următoarele clase și interfețe:

- clasa **Shape** este o **clasă abstractă** cu două metode abstracte (**arie()** și **perimetru()**)
- **Scalable** - interfață de bază ce definește o metodă **scale(double factor)**, pentru a scala o formă.
- **PrintableScalable** – interfață ce extinde **Scalable** și adaugă o metodă **printDetails()** pentru a imprima detaliile formei.
- clasele **Cerc** și **Patrat** vor extinde **Shape** și vor implementa **PrintableScalable**, oferind astfel atât calculul geometric, cât și metodele de scalare și afișare a detaliilor.

Creați un vector de obiecte de tip **Shape** și **sortați vectorul** în ordine crescătoare / descrescătoare a ariei ! Ce modificări se aduc codului dacă utilizăm interfața **Comparable**.